HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO THỰC HÀNH
DỊCH NGƯỢC
AN TOÀN GAME
BENTOLOOP

Giảng viên hướng dẫn: Cao Minh Thắng
Lớp: D24CQGA02-B

Sinh viên **MSV**
Nguyễn Thành Vinh B24DCGA162

HÀ NỘI - 10/2026

MỤC LỤC

1 Mục tiêu và môi trường thực hành 2
2 Chuẩn bị thư viện và xác định VerifyLicense 2
2.1 Decompile bằng APK Easy Tool 2
2.2 Tra cứu hàm trong dữ liệu dump 2
3 Phân tích và kiểm chứng license key 3
3.1 Theo dõi phép so sánh và giá trị trả về 3
3.2 Đọc StringLiteral 2770 và đối chiếu địa chỉ 4
3.3 Kiểm chứng trên APK gốc 5
4 Sửa hàm và thay thư viện 6
4.1 Thay hai lệnh đầu VerifyLicense 6
4.2 Xuất thư viện có chứa byte đã sửa 6
4.3 Thay thư viện trong thư mục đã Decompile 7
5 Build APK và kiểm thử bản đã sửa 8
5.1 Compile và ký bằng APK Easy Tool 8
5.2 Kiểm thử với chuỗi khác key tìm được 10
6 Nhận xét và phương án phòng tránh 11
6.1 Nguyên nhân cơ chế bị vòng tránh 11
6.2 Biện pháp giảm rủi ro 11

1 Mục tiêu và môi trường thực hành

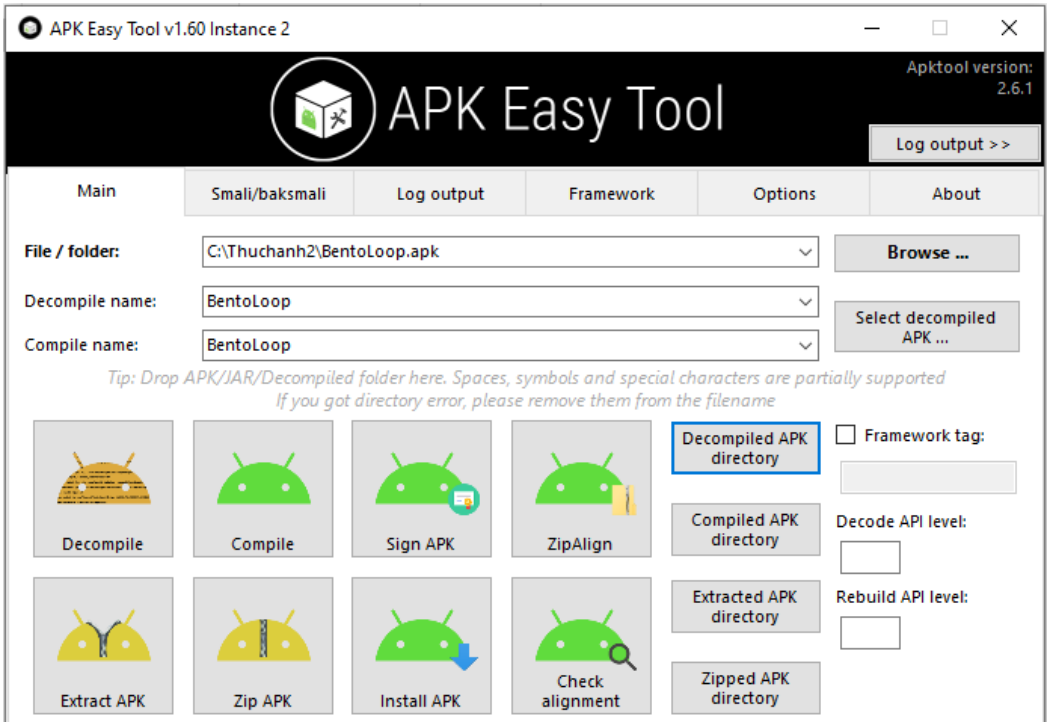
Bài thực hành phân tích cơ chế kiểm tra license của BentoLoop, tìm key hợp lệ và sửa hàm kiểm tra để build lại APK. Kết quả thu được là key BENTO-EASY-2025 được APK gốc chấp nhận; bản APK đã sửa báo kích hoạt thành công với chuỗi vinh123.

Quá trình phân tích thực hiện trên Windows trong máy ảo VMware. APK Easy Tool được dùng để Decompile, Compile và ký APK; dữ liệu dump do giảng viên cung cấp giúp tra cứu lớp và địa chỉ; Ghidra dùng để đọc và sửa mã native. Kết quả được kiểm thử trên thiết bị Android kết nối với máy thật.

2 Chuẩn bị thư viện và xác định VerifyLicense

2.1 Decompile bằng APK Easy Tool

Trong APK Easy Tool, chọn BentoLoop.apk bằng Browse rồi bấm Decompile. Công cụ tạo thư mục BentoLoop chứa các thành phần của ứng dụng. Đây là thư mục làm việc dùng cả khi lấy thư viện để phân tích và khi Compile APK sau khi sửa.



Hình 1. Chọn BentoLoop.apk và sử dụng Decompile trong APK Easy Tool

Thư viện cần phân tích là libil2cpp.so trong lib\arm64-v8a của thư mục đã Decompile. Đây là bản ARM64 chứa mã native của ứng dụng Unity IL2CPP. Thư viện này được mở trong Ghidra và cũng là file cần thay trước khi build lại APK.

2.2 Tra cứu hàm trong dữ liệu dump

Trong dump.cs, tìm LicenseVerifier thuộc namespace GeckoArrow.Runtime.LoadingSystem. Khai báo VerifyLicense(string inputKey) cho biết hàm nhận chuỗi nhập vào và trả về bool. Thân hàm rỗng trong dump chỉ phục vụ tra cứu; logic thực thi được đọc trong libil2cpp.so.

```
// Namespace: GeckoArrow.Runtime.LoadingSystem
public static class LicenseVerifier // TypeDefIndex: 12855
{
        // Fields
        private static readonly byte[] EncryptedBytes; // 0x0

        // Methods

        // RVA: 0x1E76FA0 Offset: 0x1E72FA0 VA: 0x1E76FA0
        public static bool CheckTransformedLicense(string key) { }

        // RVA: 0x1E7689C Offset: 0x1E7289C VA: 0x1E7689C
        public static bool VerifyLicense(string inputKey) { }
```

Hình 2. Khai báo LicenseVerifier và RVA của VerifyLicense trong dump.cs

RVA của VerifyLicense là 0x1E7689C. Chương trình Ghidra có Image Base 0x100000, nên địa chỉ cần mở bằng Go To là 0x1F7689C. Tên hàm và nhãn chuỗi được gắn từ script.json của cùng gói dump để thuận tiện đối chiếu mã native.

```
Địa chỉ Ghidra = Image Base + RVA
0x00100000 + 0x01E7689C = 0x01F7689C
```

3 Phân tích và kiểm chứng license key

3.1 Theo dõi phép so sánh và giá trị trả về

Mở VerifyLicense tại 0x1F7689C và theo dõi đoạn xử lý đầu vào trong Decompiler. Ba lời gọi IsNullOrWhiteSpace, Trim và Equals dẫn đến đầu mối tìm key.

```
18  }
19  uVar2 = System.String$$IsNullOrWhiteSpace(param_1,0);
20  if ((uVar2 & 1) == 0) {
21    if (param_1 == 0) {
22                    /* WARNING: Subroutine does not return */
23      FUN_01da5340();
24    }
25    uVar3 = System.String$$Trim(param_1,0);
26    uVar2 = System.String$$Equals(uVar3,StringLiteral_2770,4,0);
27    if ((uVar2 & 1) == 0) {
```

Hình 3. VerifyLicense xử lý đầu vào và so sánh với StringLiteral_2770

Trong Hình 3, dòng 19 kiểm tra chuỗi rỗng hoặc chỉ có khoảng trắng. Khi kết quả bằng 0, luồng tiếp tục tới dòng 25 để Trim khoảng trắng ở hai đầu. Dòng 26 gọi String.Equals(uVar3, StringLiteral_2770, 4, 0), so sánh chuỗi đã xử lý với dữ liệu mang nhãn StringLiteral_2770. Nhãn này biểu diễn một chuỗi, không phải tên hàm.

```
27    if ((uVar2 & 1) == 0) {
28      if (*(int *)(GeckoArrow.Runtime.LoadingSystem.LicenseVerifier_TypeInfo + 0xe4) == 0) {
29        thunk_FUN_01e01e10();
30      }
31      uVar2 = GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$CheckTransformedLicense(uVar3);
32      if ((uVar2 & 1) == 0) {
33        uVar2 = GeckoArrow.Runtime.LoadingSystem.SecurityAuthService$$DispatchVerification(uVar3);
34        if ((uVar2 & 1) == 0) goto LAB_01f76914;
35        iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
36        puVar4 = &StringLiteral_11019;
37      }
38      else {
```

Hình 4. Các dòng 27 đến 38 nối kết quả Equals với các nhánh kiểm tra bổ sung

Hình 4 bổ sung phần giữa hàm. Điều kiện ở dòng 27 mở nhánh khi Equals không khớp. Dòng 31 gọi CheckTransformedLicense; nếu kết quả vẫn bằng 0, dòng 33 gọi DispatchVerification. Dòng

34 chuyển tới LAB_01f76914 khi lần kiểm tra này cũng thất bại. Ngược lại, khi Equals đã khớp, khối kiểm tra bổ sung được bỏ qua và luồng tiếp tục tới phần trả về thành công bên dưới.

```
39          iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
40          puVar4 = &StringLiteral_11021;
41        }
42      }
43      else {
44        iVar1 = *(int *)(UnityEngine.Debug_TypeInfo + 0xe4);
45        puVar4 = &StringLiteral_11020;
46      }
47      if (iVar1 == 0) {
48        thunk_FUN_01e01e10();
49      }
50      UnityEngine.Debug$$Log(*puVar4,0);
51      uVar3 = 1;
52    }
53    else {
54  LAB_01f76914:
55      uVar3 = 0;
56    }
57    return uVar3;
58  }
59
```

Hình 5. Các nhánh cuối VerifyLicense trả về 1 khi thành công và 0 khi thất bại

Hình 5 cho thấy dòng 51 gán uVar3 = 1 và dòng 57 trả về giá trị đó. Tại LAB_01f76914, dòng 55 gán uVar3 = 0. Ghép phần giữa hàm với phần cuối xác định được đường trả về của hai trường hợp, từ đó chọn StringLiteral_2770 để đọc chuỗi được nhánh trực tiếp chấp nhận.

3.2 Đọc StringLiteral_2770 và đối chiếu địa chỉ

Bấm đúp StringLiteral_2770 trong lời gọi Equals để chuyển đến dữ liệu trong Listing. Vùng được chọn dưới đây hiển thị đồng thời nhãn chuỗi, địa chỉ và nội dung key.

```
                    StringLiteral_2770                              XREF[3]:     GeckoArrow.Runtime.LoadingSystem...
                                                                                 GeckoArrow.Runtime.LoadingSystem...
                                                                                 043e6878(*)
044fce70 a5 15 00        undefined8  00000000A00015A5h                              BENTO-EASY-2025
         a0 00 00
         00 00
```

Hình 6. StringLiteral_2770 tại 044fce70 chứa BENTO-EASY-2025

Trong Hình 6, cột địa chỉ bên trái ghi 044fce70, nhãn là StringLiteral_2770 và chú thích bên phải là BENTO-EASY-2025. Đây chính là nhãn được truyền vào Equals ở Hình 3, nên chuỗi này được chọn để kiểm chứng trên APK gốc.

```
{
  "value": "BENTO-EASY-2025",
  "address": "0x43FCE70"
},
```

Hình 7. stringliteral.json ghi cùng key với address 0x43FCE70

Hình 7 là mục tương ứng trong stringliteral.json: value bằng BENTO-EASY-2025 và address bằng 0x43FCE70. Địa chỉ JSON cộng Image Base 0x100000 được 0x044FCE70, khớp vị trí trong Listing. Hai nguồn dữ liệu vì thế cùng xác nhận một chuỗi và một vị trí sau quy đổi.

Memory Map - Image Base: 00100000

Name	Start	End
segment_1.1	00100000	0010026f
.note.android.ident	00100270	00100307

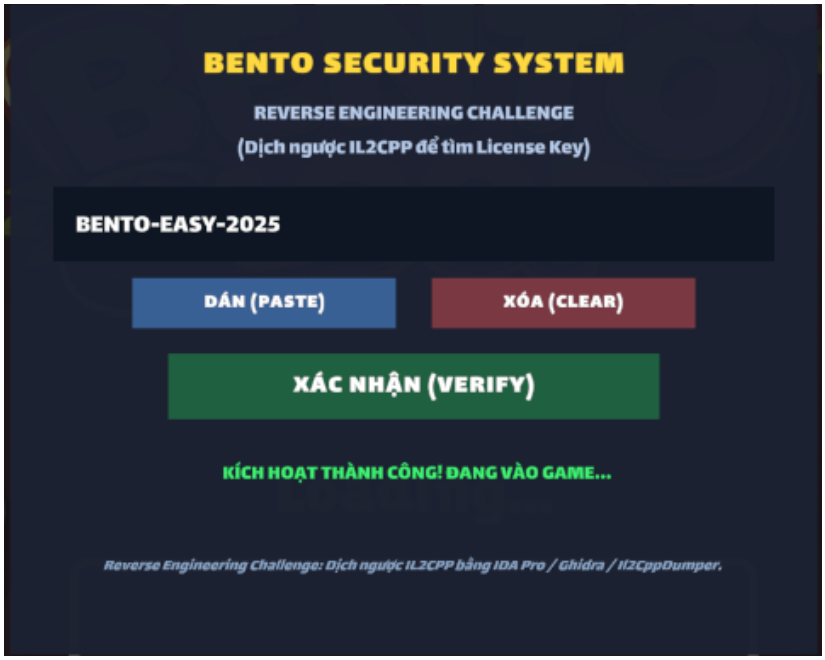
Hình 8. Memory Map hiển thị Image Base 00100000

Thanh tiêu đề Memory Map trong Hình 8 ghi Image Base: 00100000, tương ứng 0x100000. Đây là giá trị dùng trong phép quy đổi địa chỉ JSON sang địa chỉ Listing, cũng là giá trị đã dùng để mở VerifyLicense ở mục 2.2.

```
0x043FCE70 + 0x00100000 = 0x044FCE70
```

3.3 Kiểm chứng trên APK gốc

Trên APK gốc, nhập BENTO-EASY-2025 rồi bấm XÁC NHẬN (VERIFY). Ảnh dưới ghi lại chuỗi đã nhập cùng thông báo KÍCH HOẠT THÀNH CÔNG! ĐANG VÀO GAME...



Hình 9. APK gốc chấp nhận key BENTO-EASY-2025



Hình 10. APK gốc chuyển vào Level 1 sau khi xác nhận key

Thông báo kích hoạt và màn chơi Level 1 xác nhận key tìm được bằng phân tích tĩnh có hiệu lực trên APK gốc. Kết quả này hoàn thành phần tìm key và là mốc đối chiếu với bản APK sửa ở phần 5.

4 Sửa hàm và thay thư viện

4.1 Thay hai lệnh đầu VerifyLicense

Lưu chương trình Ghidra thành bản sao libil2cpp_patched rồi mở VerifyLicense tại 0x1F7689C. Hai lệnh đầu của hàm được chọn để sửa bằng Patch Instruction.

```
01f7689c fe 0f 1e f8     str        x30,[sp, #-0x20]!
01f768a0 f4 4f 01 a9     stp        x20,x19,[sp, #local_10]
01f768a4 54 38 01 d0     adrp       x20,0x4680000
```

Hình 11. Hai lệnh đầu VerifyLicense trước khi sửa

Hình 11 ghi lại lệnh str x30, [sp, #-0x20]! tại 0x1F7689C và stp x20, x19, [sp, #0x10] tại 0x1F768A0. Đây là các lệnh chuẩn bị stack trước khi thực hiện phần kiểm tra license.

```
01f7689c 20 00 80 52     mov        w0,#0x1
01f768a0 c0 03 5f d6     ret
```

Hình 12. Hai lệnh sau khi sửa là mov w0, #0x1 và ret

Tại 0x1F7689C, thay str bằng mov w0, #0x1; tại 0x1F768A0, thay stp bằng ret. Hình 12 hiển thị các byte mới lần lượt là 20 00 80 52 và C0 03 5F D6. Lệnh mov đặt giá trị trả về bằng 1, còn ret kết thúc hàm ngay. Hai lệnh dài tổng cộng 8 byte.

Decompile: GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$VerifyLicense - (libil2...

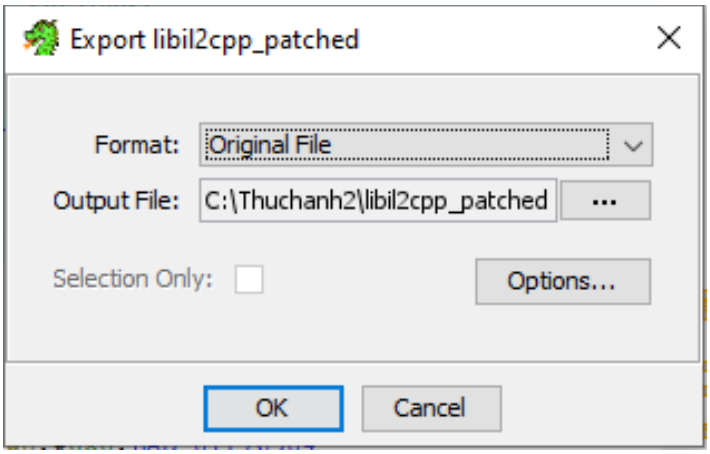
```
1
2 undefined8 GeckoArrow.Runtime.LoadingSystem.LicenseVerifier$$VerifyLicense(void)
3
4 {
5   return 1;
6 }
7
```

Hình 13. Decompiler của VerifyLicense sau sửa chỉ còn return 1

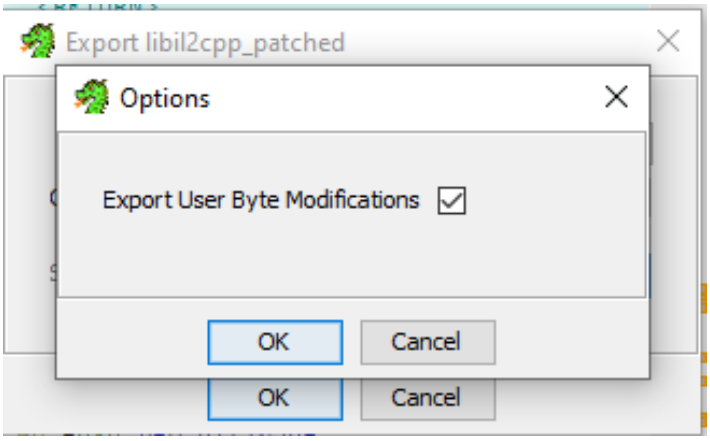
Hình 13 cho thấy thân VerifyLicense đã rút gọn thành return 1. Từ điểm vào hàm, mã mới trả về thành công trước khi đi tới Trim, Equals hoặc các nhánh kiểm tra bổ sung. Hai lệnh chuẩn bị stack ban đầu đã được thay, nên việc trả về sớm không cần khôi phục stack do phần thân cũ tạo ra.

4.2 Xuất thư viện có chứa byte đã sửa

Mở File → Export Program, chọn Format là Original File và đặt file đích C:\Thuchanh2\libil2cpp_patched. Bước Export tạo file thư viện để đưa vào APK; thao tác Save trong project chỉ lưu cơ sở dữ liệu phân tích của Ghidra.

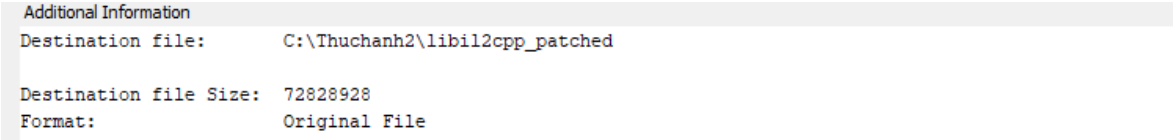


Hình 14. Xuất chương trình với định dạng Original File



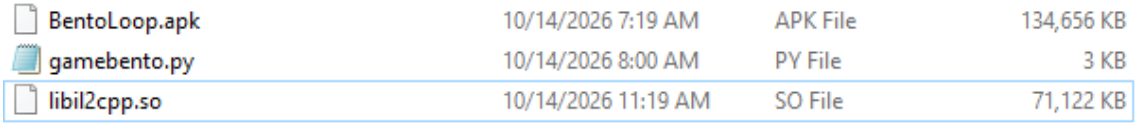
Hình 15. Bật Export User Byte Modifications để đưa byte đã sửa vào file xuất

Trong Options, tích Export User Byte Modifications rồi xác nhận xuất file. Tùy chọn này giữ phần byte đã patch trong kết quả. File xuất được đổi tên thành libil2cpp.so để khớp tên thư viện mà ứng dụng sử dụng.



Hình 16. Export Results Summary ghi file đích và định dạng Original File

Hình 16 là kết quả sau khi xuất: Destination file trỏ tới C:\Thuchanh2\libil2cpp_patched và Format là Original File. Ảnh này kiểm chứng bước Export đã tạo file đích, thay vì chỉ lưu thay đổi trong project Ghidra.

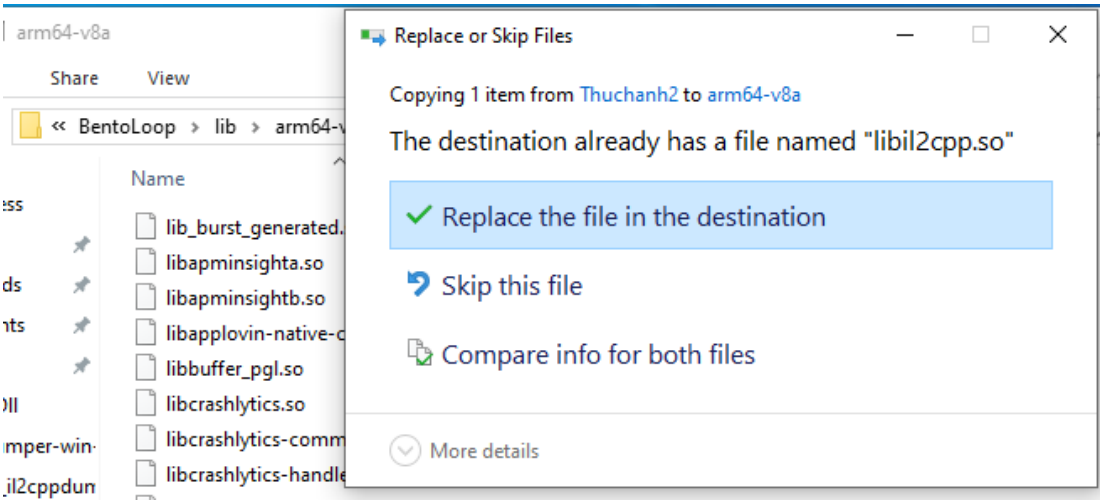


Hình 17. File thư viện mang tên libil2cpp.so trước khi copy

Hình 17 ghi lại file dưới tên libil2cpp.so trước khi copy. Tên này khớp thư viện trong lib\arm64-v8a; hộp thoại thay file ở bước tiếp theo ghi rõ cùng tên và đúng thư mục đích.

4.3 Thay thư viện trong thư mục đã Decompile

Copy libil2cpp.so đã xuất vào lib\arm64-v8a của thư mục BentoLoop đã Decompile. Khi Windows báo trùng tên, chọn Replace the file in the destination để thay đúng thư viện cũ.



Hình 18. Thay libil2cpp.so trong thư mục arm64-v8a

Sau bước này, thư mục BentoLoop chứa thư viện ARM64 đã sửa và được dùng làm đầu vào Compile. Địa chỉ patch và các lệnh trong báo cáo áp dụng cho bản thư viện đã phân tích.

5 Build APK và kiểm thử bản đã sửa

5.1 Compile và ký bằng APK Easy Tool

Trong APK Easy Tool, dùng Select decompiled APK để chọn thư mục BentoLoop chứa apktool.yml, sau đó bấm Compile. Công cụ đóng gói lại các thành phần cùng libil2cpp.so đã thay và tạo APK để ký, cài đặt.

5.1.1 Lỗi manifest khi dùng Apktool cũ

Lần Compile với Apktool 2.6.1 dừng ở manifest. Log dưới đây chỉ ra lỗi unexpected element <property>, vì công cụ build chưa xử lý được phần tử này trong APK.

```
error: unexpected element <property> found in <manifest><application><activity>.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:270:
error: unexpected element <property> found in <manifest><application><activity>.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:273:
error: unexpected element <property> found in <manifest><application><activity>.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:276:
error: unexpected element <property> found in <manifest><application><activity>.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:279:
error: unexpected element <property> found in <manifest><application><activity>.
W: C:\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop\AndroidManifest.xml:282:
error: unexpected element <property> found in <manifest><application><activity>.
brut.androlib.AndrolibException: brut.common.BrutException: could not exec (exit code = 1):
```

Hình 19. Log build báo unexpected element property trong AndroidManifest.xml

Các dòng lỗi trong Hình 19 đều trỏ tới AndroidManifest.xml và phần tử property. Bước xử lý là thêm Apktool 2.12.1 vào APK Easy Tool rồi chọn phiên bản mới để Compile lại. Lỗi này thuộc khâu build tài nguyên, chưa phải kết quả chạy VerifyLicense đã sửa.

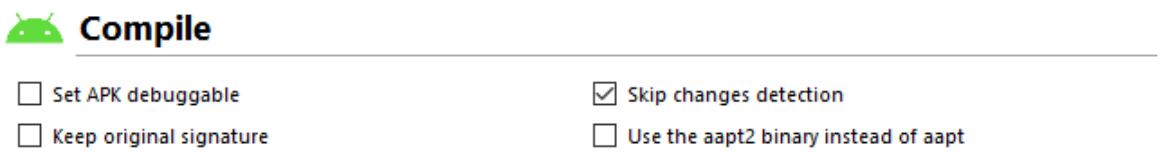
5.1.2 Bỏ tham số không còn được hỗ trợ

```
Compiling APK file... (Press ESC to cancel)
Command: java.exe -jar "C:\Thuchanh2\apk-easy-tool-1-60\Apktool\apktool_2.12.1.jar" b -f --
use-aapt2 -o "C:\Users\DucNM\Documents\APK Easy Tool\2-Recompiled APKs\BentoLoop.apk" "C:
\Users\DucNM\Documents\APK Easy Tool\1-Decompiled APKs\BentoLoop"

Unrecognized option: --use-aapt2
Apktool 2.12.1 - a tool for reengineering Android apk files
```

Hình 20. Apktool 2.12.1 báo Unrecognized option với use aapt2

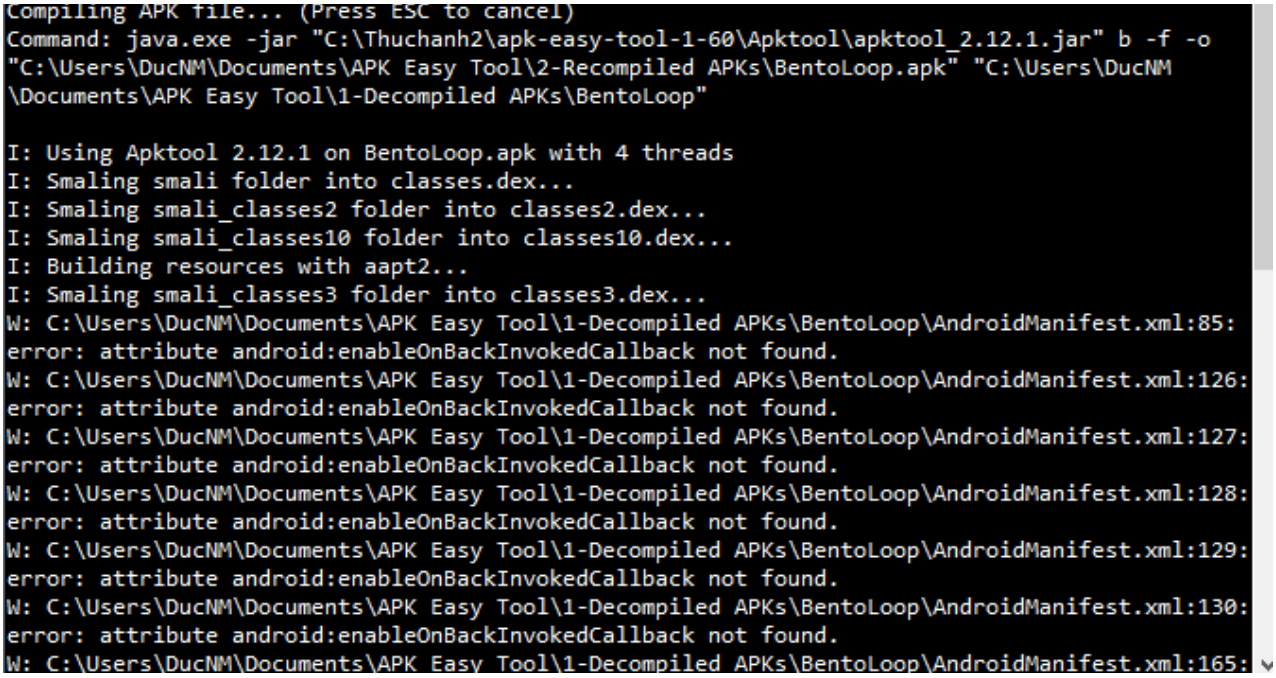
Hình 20 cho thấy lệnh build đã gọi apktool_2.12.1.jar nhưng vẫn kèm --use-aapt2, sau đó báo Unrecognized option. Cần chỉnh tùy chọn trong giao diện APK Easy Tool để bỏ tham số cũ này.



Hình 21. Bỏ tích Use the aapt2 binary instead of aapt trong phần Compile

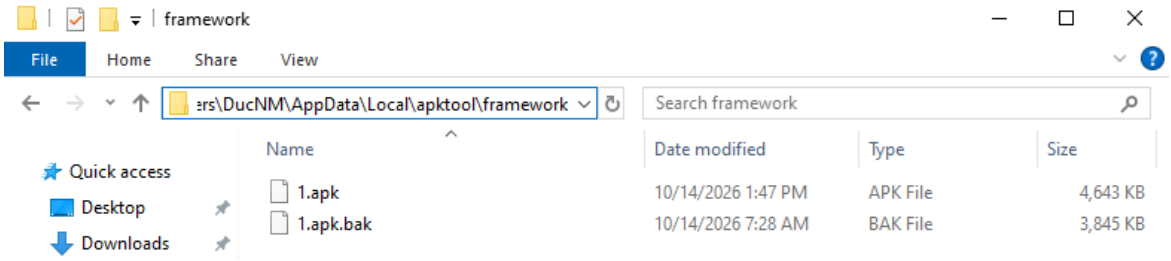
Trong Hình 21, ô Use the aapt2 binary instead of aapt không được tích. Apktool mới dùng AAPT2 mặc định, nên bỏ ô này giúp giao diện ngừng thêm --use-aapt2 vào lệnh build. Các tùy chọn khác không cần thay để xử lý lỗi đang gặp.

5.1.3 Làm mới framework trước khi Compile lại



Hình 22. Build tiếp tục nhưng báo thiếu enableOnBackInvokedCallback

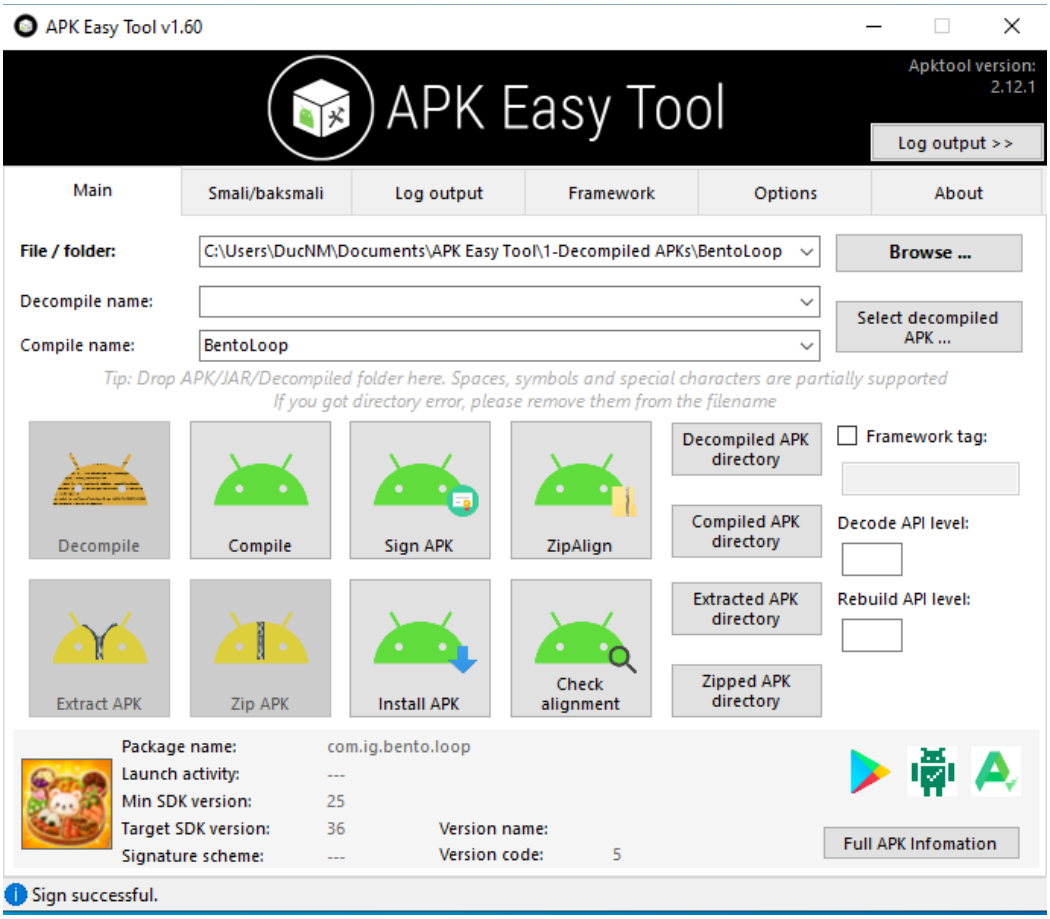
Hình 22 hiển thị Using Apktool 2.12.1 và Building resources with aapt2, xác nhận quá trình đã qua lỗi tham số. Các dòng tiếp theo báo attribute android:enableOnBackInvokedCallback not found. Framework đang được lưu trong bộ nhớ đệm cần được làm mới để nhận thuộc tính này.



Hình 23. Framework mới 1.apk nằm cạnh bản sao cũ 1.apk.bak

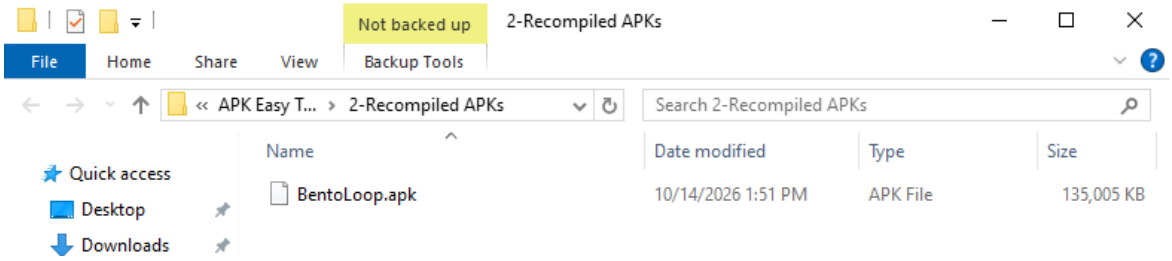
Đổi tên framework\1.apk trong bộ nhớ đệm Apktool thành 1.apk.bak rồi Compile lại. Hình 23 cho thấy file 1.apk mới và bản sao 1.apk.bak cùng nằm trong thư mục framework, có thời điểm sửa khác nhau. Sau bước này, công cụ build và ký được APK như ảnh kết quả bên dưới.

5.1.4 Kết quả build và ký APK



Hình 24. APK Easy Tool báo Sign successful sau khi build và ký

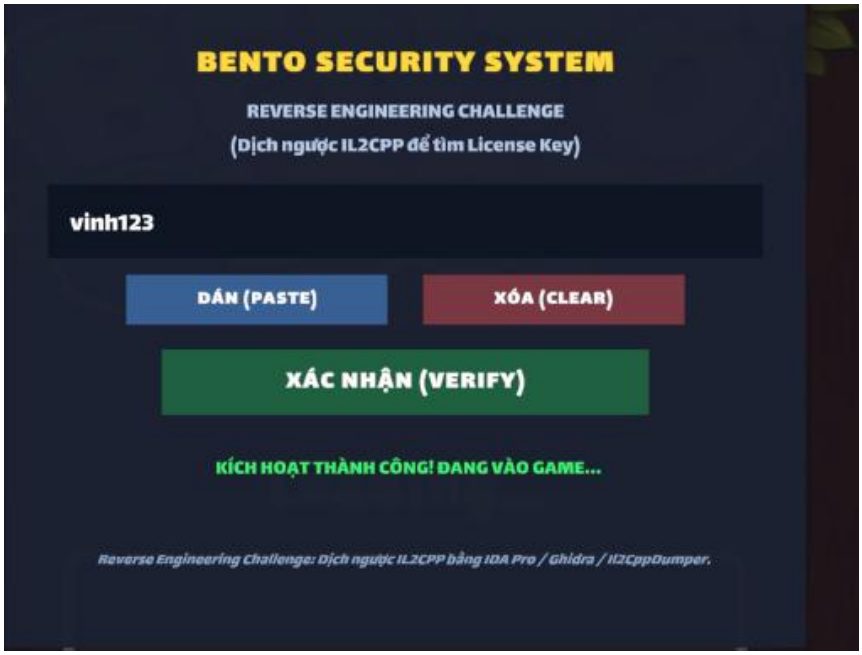
Mở Compiled APK directory để lấy BentoLoop.apk trong thư mục 2-Recompiled APKs. File này là đầu ra của bước build lại và được chuyển từ máy ảo sang máy thật để cài trên thiết bị Android.



Hình 25. BentoLoop.apk được tạo trong thư mục 2 Recompiled APKs

5.2 Kiểm thử với chuỗi khác key tìm được

Trên APK mới, nhập vinh123 rồi bấm XÁC NHẬN (VERIFY). Chuỗi này khác BENTO-EASY-2025 nhưng ứng dụng vẫn hiển thị KÍCH HOẠT THÀNH CÔNG! ĐANG VÀO GAME...



Hình 26. APK đã sửa báo kích hoạt thành công với chuỗi vinh123

Kết quả phù hợp với mã VerifyLicense đã sửa để trả về 1 ngay khi được gọi. Ảnh này ghi nhận bản APK mới chấp nhận một chuỗi khác key; ảnh kiểm chứng APK gốc ở mục 3.3 ghi nhận key hợp lệ và màn chơi Level 1.

Bản ứng dụng	Chuỗi thử nghiệm	Kết quả ghi nhận
APK gốc	BENTO-EASY-2025	Kích hoạt thành công và vào Level 1
APK đã sửa	vinh123	Kích hoạt thành công và đang vào game

6 Nhận xét và phương án phòng tránh

6.1 Nguyên nhân cơ chế bị vòng tránh

Key được lưu dưới dạng chuỗi trong thư viện phía client và được so sánh trực tiếp trong VerifyLicense. Sau khi xác định nhãn StringLiteral_2770, nội dung BENTO-EASY-2025 có thể đọc từ Listing và đối chiếu với JSON. Các nhánh kiểm tra bổ sung không ngăn được việc tìm key qua nhánh so sánh trực tiếp đang tồn tại.

Quyết định chấp nhận license còn phụ thuộc vào giá trị trả về của một hàm trong mã client. Thay 8 byte đầu hàm đã đưa luồng thực thi đến return 1. APK sau khi build và ký lại vẫn báo kích hoạt thành công với vinh123, cho thấy cơ chế đã quan sát có thể bị vòng tránh bằng cách sửa thư viện.

6.2 Biện pháp giảm rủi ro

Với ứng dụng có máy chủ, xác minh license theo tài khoản hoặc quyền sử dụng trên máy chủ và tiếp tục kiểm tra quyền tại các thao tác cung cấp dữ liệu, dịch vụ. Quyền truy cập khi đó không chỉ phụ thuộc vào một hàm hoặc một màn hình trong APK.

Loại bỏ key chung và nhánh chấp nhận thử nghiệm khỏi bản phát hành. Nếu cần license offline, có thể dùng license ký số: client giữ khóa công khai để xác minh, còn khóa riêng phát hành license được lưu ngoài ứng dụng. Cách này hạn chế việc nhúng một bí mật chung có thể đọc trực tiếp từ APK, nhưng phần kiểm tra trong client vẫn cần được bảo vệ trước can thiệp.

Kết hợp kiểm tra chữ ký ứng dụng, tính toàn vẹn thư viện và làm rối mã để tăng khả năng phát hiện sửa đổi và tăng chi phí phân tích. Các biện pháp này bổ sung cho cơ chế xác minh quyền, không bảo đảm riêng mã client sẽ ngăn được mọi cách patch.

Tài liệu tham khảo

Apktool — Tham số build: https://apktool.org/docs/2.x/cli-parameters/

Apktool — Quản lý framework: https://apktool.org/wiki/in-depth/frameworks/

OWASP MASVS RESILIENCE 2 — Bảo vệ trước can thiệp mã: https://mas.owasp.org/MASVS/controls/MASVS-RESILIENCE-2/